

Week 41, Tuesday Session

Flipped classroom: your questions, then one case – the feed-forward part of a neural network, step by step

Bring your laptop; open `week41tuesday.ipynb` and `exercisesweek41.ipynb`

Morten Hjorth-Jensen

Department of Physics and Center for Computing in Science Education,
University of Oslo, Norway

How today works

The format (as every Tuesday)

Mondays are lectures; Tuesdays belong to your questions and to concrete cases. Today there is *one* case, and it is the skeleton of this week's exercises.

1. **First ~20 min:** your open questions from Monday's lecture – the four equations of backpropagation and the sum over k , why the output error is $a^L - y$ for all three heads, the ℓ_2 and ℓ_1 penalties, the code and its gradient check, the MNIST run – discussed in plenum.
2. **Rest of the double session:** Case 1, worked in groups with `week41tuesday.ipynb`: the feed-forward pass of a fully connected network in six steps – one layer by hand with the shapes, a list of layers, an activation per layer, many samples at once with the transposed convention, a prediction on the digits data before any training, and (optional) the cost with `jax.grad`. Then the insight questions in plenum.
3. **Last 5 min:** wrap-up, the exercises, and what Monday builds on.

Exercises week 41

The same six steps on the iris data, in `exercisesweek41.ipynb`. Deadline **Friday October 9 at midnight**. Project 2 is published after the Project 1 deadline and builds on exactly this code.

Picking up from Monday

Monday in one sentence each

The error of a node is $\delta = \partial C / \partial z$; a weight's gradient is the error at the node it arrives at times the signal at the node it leaves. The four equations: start at the output, pull the error back through the transposed weights and modulate by f' (the sum over the receiving nodes is the delta @ W.T the template already had), then the bias and weight gradients. Two forward passes buy the whole gradient; finite differences check it. The loss follows the distribution of the target, the output activation its range, and for linear + squared error, sigmoid + cross entropy and softmax + multiclass cross entropy the output error is $a^L - y$. ℓ_2 decays weights, ℓ_1 zeroes them. The template with a list of layers reads 70000 digits at 97.6%.

Questions we expect

Why is δ both the bias gradient and the adjoint of z ? Why does the softmax need the full Jacobian and the sigmoid not? Why were the penalties only a cost on the Runge function, and when would they help? Why did the sigmoid network collapse at $\gamma = 0.5$? Why is $1/n$ in the output error and nowhere else? Why do the output errors of the softmax sum to zero? Ask.

Today's single question

Before the backward pass can exist there must be a forward pass that keeps its shapes straight. Today you write one from scratch – twice, in the two weight conventions – and run it on real data *before* any training, so that you see what an untrained network does.

Case 1: the feed-forward pass, six steps on the digits data

Goal: leave with your own `feed_forward` (single vector and batched) running on the 1797 digits of `scikit-learn`, and with the shapes of every object written down (notebook, Case 1; seed 2026). The exercises are the same steps on the iris data.

1. **One layer by hand:** x with 64 features, W^1 of shape (8, 64), b^1 ; $z^1 = W^1x + b^1$, $a^1 = \text{ReLU}(z^1)$. Write every shape down *before* running.
2. **A list of layers:** `create_layers(64, [16, 8, 10])` returns (W, b) pairs; `feed_forward_all_relu` loops over them. Then remove the activations: three layers equal one.
3. **An activation per layer:** pass `[sigmoid, ReLU, softmax_vec]` – the function objects, not their values. What changes with ReLU at the output?
4. **Many samples at once:** samples in rows, W of shape (inputs, outputs), $a @ W + b$ – Eq. (8.21), Monday's convention. Check row 0 against the single-vector pass.
5. **Predict before training:** 64–32–10, sigmoid and softmax, all 1797 digits; accuracy of five freshly drawn networks.
6. **Optional:** the multiclass cross entropy, `jax.grad` with respect to the list of layers, 300 steps of gradient descent.

Checkpoints (plenary after ~40 min)

Step 2: 1266 parameters; three linear layers and one agree to every digit. Step 4: softmax rows sum to 1; batched row 0 against the single pass 10^{-17} . Step 5: accuracies 0.0996, 0.0996, 0.1002, 0.0968, 0.1091 – guessing; mean largest probability 0.17; almost every digit called “8” or “6”. Step 6: cost $2.28 \rightarrow 0.15$, accuracy $0.11 \rightarrow 0.97$ on the training set in 300 steps at $\gamma = 1$.

Case 1: the code you should end up with

```
def create_layers(network_input_size, layer_output_sizes, rng):
    """W of shape (outputs, inputs): one vector at a time."""
    layers = []
    i_size = network_input_size
    for layer_output_size in layer_output_sizes:
        W = rng.normal(0, 1/np.sqrt(i_size), (layer_output_size, i_size))
        b = np.zeros(layer_output_size) + 0.01
        layers.append((W, b))
        i_size = layer_output_size
    return layers

def feed_forward(x, layers, activation_funcs):
    a = x
    for (W, b), activation_func in zip(layers, activation_funcs):
        z = W @ a + b
        a = activation_func(z)
    return a
```

```
def create_layers_batch(network_input_size, layer_output_sizes, rng):
    """W of shape (inputs, outputs): samples in rows, Eq. (8.21)."""
    ... W = rng.normal(0, 1/np.sqrt(i_size), (i_size, layer_output_size))
    )

def feed_forward_batch(inputs, layers, activation_funcs):
    a = inputs
    for (W, b), activation_func in zip(layers, activation_funcs):
        z = a @ W + b # one product, one broadcast
        a = activation_func(z)
    return a
```

```
def softmax(z): # rows are samples
    e_z = jnp.exp(z - jnp.max(z, axis=1, keepdims=True))
    return e_z/jnp.sum(e_z, axis=1, keepdims=True)

layers = create_layers_batch(64, [32, 10], rng)
predictions = feed_forward_batch(inputs, layers, [sigmoid, softmax])
print(accuracy(predictions, targets)) # ~0.10

def cost(inputs, layers, activation_funcs, target):
    predict = feed_forward_batch(inputs, layers, activation_funcs)
    return -jnp.mean(jnp.sum(target*jnp.log(predict + 1e-12), axis=1))

gradient_func = jax.grad(cost, argnums=1) # w.r.t. the layers
layers_grad = gradient_func(inputs, layers, [sigmoid, softmax],
                             targets)

for epoch in range(300): # Eq. (8.70)
    layers_grad = gradient_func(inputs, layers, [sigmoid, softmax],
                                targets)
    layers = [(W - gamma*dW, b - gamma*db)
              for (W, b), (dW, db) in zip(layers, layers_grad)]
```

Three things to get right

Shapes: $(8, 64) \times (64,) \rightarrow (8,)$ one vector at a time;
 $(1797, 64) \times (64, 8) \rightarrow (1797, 8)$ for all samples – the second W is the transpose of the first. **Functions as values:** the list holds sigmoid, not sigmoid(z). **jax.numpy everywhere** in the functions you will differentiate, and the softmax on rows with axis=1, keepdims=True – the version with axis=0 is the classic bug.

Case 1: what you should find

Steps 1–4: the shapes

$z^1 =$
(0.507, 0.150, -0.206, -0.195, -0.545, -0.005, 0.575, 0.453)
and ReLU zeroes four of the eight. The 64–16–8–10 network has $16 \cdot 64 + 16 + 8 \cdot 16 + 8 + 10 \cdot 8 + 10 = 1266$ parameters. Without activations, $W^3 W^2 W^1$ and $W^3(W^2 b^1 + b^2) + b^3$ reproduce the three-layer output exactly: depth without non-linearity is one linear map. With [sigmoid, ReLU, softmax_vec] the output sums to 1; with ReLU last it sums to 3.3 and has exact zeros. Batched row 0 and the single-vector pass with the transposed weights agree to 1.4×10^{-17} .

Step 5: the untrained network

Five fresh 64–32–10 networks: accuracy 0.0996, 0.0996, 0.1002, 0.0968, 0.1091 – guessing on ten classes. The largest probability per row averages 0.17 (a flat softmax), and the first twenty digits are all called “8” or “6”: random weights give the same score pattern for every image.

Step 6: the gradient from the forward pass alone

`jax.grad(cost, argnums=1)` returns a list of (dW, db) with exactly the shapes of the layers: (64, 32), (32,), (32, 10), (10,). Plain gradient descent on all 1797 samples, $\gamma = 1$:

step	cost	accuracy
0	2.282	0.106
50	0.927	0.882
100	0.436	0.930
200	0.217	0.953
299	0.154	0.969

Training-set accuracy only – no test set, no minibatches, one γ – so it shows that the gradient points downhill and nothing about generalisation. Monday’s code did the honest version on MNIST. The exercises: the same six steps with 4 features, 8 hidden nodes and 3 classes, where guessing gives 33%.

Case 1: do you have the insight?

1. Three layers without activation functions gave the same output as one layer with $\tilde{W} = W^3 W^2 W^1$. Which property of linear maps is responsible, and what does the universal approximation theorem (Section 8.4) require of the activation to avoid it?
2. The single-vector convention stores W as (outputs, inputs) and computes Wx ; the batched one stores (inputs, outputs) and computes XW . Why does putting the samples in rows force the transpose, and which convention does Monday's backward pass ($\text{delta} @ W.T, a.T @ \text{delta}$) assume?
3. The untrained networks scored 10%, the largest probability per row averaged 0.17, and almost every digit was called "8" or "6". Why is that what random weights produce? What would the output look like with ReLU or the sigmoid at the output, and why does the multiclass cross entropy need the softmax?
4. We drew the weights with scale $1/\sqrt{\text{inputs}}$. With $\mathcal{N}(0, 1)$ weights and 64 inputs, how large is z^1 for a sigmoid hidden layer, what becomes of $\sigma'(z^1)$, and what does Eq. (8.66) then deliver to the first layer?

Case 1: do you have the insight?

1. Three layers without activation functions gave the same output as one layer with $\tilde{W} = W^3 W^2 W^1$. Which property of linear maps is responsible, and what does the universal approximation theorem (Section 8.4) require of the activation to avoid it?
2. The single-vector convention stores W as (outputs, inputs) and computes Wx ; the batched one stores (inputs, outputs) and computes XW . Why does putting the samples in rows force the transpose, and which convention does Monday's backward pass ($\text{delta} @ W.T, a.T @ \text{delta}$) assume?
3. The untrained networks scored 10%, the largest probability per row averaged 0.17, and almost every digit was called "8" or "6". Why is that what random weights produce? What would the output look like with ReLU or the sigmoid at the output, and why does the multiclass cross entropy need the softmax?
4. We drew the weights with scale $1/\sqrt{\text{inputs}}$. With $\mathcal{N}(0, 1)$ weights and 64 inputs, how large is z^1 for a sigmoid hidden layer, what becomes of $\sigma'(z^1)$, and what does Eq. (8.66) then deliver to the first layer?

What we hope you say

(1) Matrix products are matrices: the composition of linear maps is linear, so depth without non-linearity adds parameters but no functions; the theorem needs a non-polynomial activation. (2) A row vector times a matrix is $x^T W$, the transpose of $W^T x$; Monday's code assumes the batched convention throughout. (3) The ten scores differ by small random amounts that hardly depend on the image, so the softmax is nearly flat and the winner is the node with the largest random bias-plus-weights, the same for most images. ReLU gives non-negative numbers with exact zeros that do not sum to one, the sigmoid ten independent numbers in $(0, 1)$; only the softmax is a distribution, and $-\ln a_{k*}$ needs a probability of the correct class. (4) z^1 sums 64 terms of order one, standard deviation $\approx 8 \text{std}(x)$: most units sit where $\sigma' \approx 0$, Eq. (8.66) multiplies the error by that σ' , and the first layer receives almost no gradient – the reason for Section 8.10.

Wrap-up, the exercises, and Monday

What you should own now

A forward pass you wrote yourself, in both weight conventions, with the shapes of every object in your head; the knowledge that an untrained network guesses, that a network without activations is linear regression in disguise, and that the softmax is what turns scores into a distribution. Everything Monday adds – the backward pass, the heads, the penalties – sits behind this forward pass.

Exercises week 41 – deadline Friday October 9 at midnight

`exercisesweek41.ipynb`: the six steps on the iris data (4 features, 3 classes), with the solution checks built into the notebook, and the optional training with `jax.grad` at the end. Hand in one notebook per group in Canvas; list your collaborators. Finish the self-check table at the end before you hand in.

Next Monday, week 42

The backward pass in your own code (the exercises of week 42 implement Eqs. (8.65)–(8.68) and compare with `jax.grad`), training with the optimisers of week 38 on minibatches, a train/test split, and the practical side of Sections 8.9–8.12: activation functions and initialisation measured, dropout, batch normalisation, hyperparameter search. Project 2 is published after the Project 1 deadline.